MLA0303_RL_PROGRAM_8 — Program with Output

Python Source Code

```
"""
Experiment 8: Monte Carlo prediction and control for a robot vacuum
cleaner learning an efficient cleaning policy while minimizing energy use.
"""

import random
from collections import defaultdict

GRID = 4
DIRTY_CELLS = {(0, 0), (1, 3), (3, 1), (2, 2)}
DOCK = (3, 3)
ACTIONS = ["UP", "DOWN", "LEFT", "RIGHT"]
DELTA = {"UP": (-1, 0), "DOWN": (1, 0), "LEFT": (0, -1), "RIGHT": (0, 1)}
GAMMA = 0.95
EPISODES = 3000
EPSILON = 0.2


def step(state):
    pos, dirty = state
    return pos, dirty


def take_action(state, action):
    pos, dirty = state
    dr, dc = DELTA[action]
    nxt = (pos[0] + dr, pos[1] + dc)
    if not (0 <= nxt[0] < GRID and 0 <= nxt[1] < GRID):
        nxt = pos
        reward = -3
    else:
        reward = -1        # energy cost per move
    new_dirty = dirty - {nxt} if nxt in dirty else dirty
    if nxt in dirty:
        reward += 10       # cleaned a dirty cell
    done = len(new_dirty) == 0 and nxt == DOCK
    if done:
        reward += 15
    return (nxt, new_dirty), reward, done


def generate_episode(Q, epsilon):
    pos = (0, 3)
    dirty = frozenset(DIRTY_CELLS)
    state = (pos, dirty)
    episode = []
    for _ in range(80):
        if random.random() < epsilon or (state not in Q):
            action = random.choice(ACTIONS)
        else:
            action = max(ACTIONS, key=lambda a: Q[state][a])
        next_state, reward, done = take_action(state, action)
        episode.append((state, action, reward))
        state = next_state
        if done:
            break
    return episode


def monte_carlo_control():
    Q = defaultdict(lambda: {a: 0.0 for a in ACTIONS})
    N = defaultdict(lambda: {a: 0 for a in ACTIONS})
    total_rewards = []
    for ep in range(EPISODES):
        episode = generate_episode(Q, EPSILON)
        G = 0
        visited = set()
        for t in reversed(range(len(episode))):
            s, a, r = episode[t]
            G = GAMMA * G + r
            if (s, a) not in visited:
                visited.add((s, a))
                N[s][a] += 1
                Q[s][a] += (G - Q[s][a]) / N[s][a]
        total_rewards.append(sum(r for _, _, r in episode))
    return Q, total_rewards
if __name__ == "__main__":
    random.seed(5)
```

```
Q, rewards = monte_carlo_control()
print(f"Average episode return (first 50): {sum(rewards[:50]) / 50:.2f}")
print(f"Average episode return (last 50) : {sum(rewards[-50:]) / 50:.2f}")

# Greedy evaluation run
pos = (0, 3)
dirty = frozenset(DIRTY_CELLS)
state = (pos, dirty)
path = [pos]
for _ in range(30):
    action = max(ACTIONS, key=lambda a: Q[state][a])
    state, _, done = take_action(state, action)
    path.append(state[0])
    if done:
        break
print("Learned cleaning path:", path)
```

Execution Output

```
Average episode return (first 50): -93.10
Average episode return (last 50) : 35.78
Learned cleaning path: [(0, 3), (1, 3), (1, 2), (2, 2), (1, 2), (1, 1), (0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (3, 1),

STDERR:
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/patches/warm_spreadsheet_runtime_on_startup.py", line
  File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/spreadsheet_warmup.py", line 772, in warm_spreadsheet_
  File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/rpc/connection.py", line 37, in get_or_create_client
  File "/tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/rpc/daemon.py", line 124, in start_daemon
TimeoutError: Timed out waiting for artifact tool daemon socket. Set ARTIFACT_TOOL_RPC_DAEMON_STARTUP_TIMEOUT_S=<seconds
```